

The Role of Randomness in Machine Learning

CSI 4106 — Introduction to Artificial Intelligence

Marcel Turcotte

2026-09-20

Introduction

Randomness plays several roles in machine learning. It is used to initialize model parameters, shuffle data, create mini-batches, construct training and test sets, and implement randomized algorithms. These roles are related, but not identical. For example, random initialization breaks the symmetry between neurons in the same layer of a neural network; if their weights were initialized identically, they could learn the same features. Random shuffling and sampling expose an algorithm to different selections and orderings of the available data.

Pseudo-Random Numbers

Most computational libraries that generate random numbers actually use *pseudo-random* number generators. They produce sequences with useful statistical properties, but the sequences are generated deterministically.

A pseudo-random number generator maintains an internal **state**. Each draw produces a value and updates this state. A **seed** initializes the state; using the same generator, the same seed, and the same sequence of calls reproduces the same values. The value 42 is conventional in examples, but it has no special statistical property.

In the following example using Python's built-in [random module](#), we generate two sequences. Because we do not reset the seed between them, the second sequence continues from the state left by the first. The values will usually change if this cell is executed in a new Python session.

```
import random

sequence_1 = [random.randint(1, 100) for _ in range(5)]
sequence_2 = [random.randint(1, 100) for _ in range(5)]
```

```
print(f"Sequence 1: {sequence_1}")
print(f"Sequence 2: {sequence_2}")
```

Sequence 1: [35, 21, 31, 41, 86]
Sequence 2: [97, 97, 28, 69, 46]

By contrast, the next example resets the generator to the same state before producing each of the first two sequences. A different seed normally selects a different sequence.

```
random.seed(42)
sequence_1 = [random.randint(1, 100) for _ in range(5)]

random.seed(42)
sequence_2 = [random.randint(1, 100) for _ in range(5)]

random.seed(123)
sequence_3 = [random.randint(1, 100) for _ in range(5)]

print(f"Sequence 1: {sequence_1}")
print(f"Sequence 2: {sequence_2}")
print(f"Sequence 3: {sequence_3}")
```

Sequence 1: [82, 15, 4, 95, 36]
Sequence 2: [82, 15, 4, 95, 36]
Sequence 3: [7, 35, 12, 99, 53]

The ability to reproduce random choices is critical in scientific computing and machine learning. It helps us debug code, compare methods under the same conditions, and communicate experiments precisely.

Python's `random` module and scikit-learn do not share a single generator. Calling `random.seed(42)` controls functions from the `random` module. In scikit-learn, an integer supplied through a parameter such as `random_state=42` controls the random choices made by that particular object or function.

Imagine that a model behaves unexpectedly and you ask a teammate to investigate. If each execution creates a different data partition, your teammate may not observe the same behaviour. Fixing the seed allows both of you to reproduce the same random choices and examine the same experiment. It does not make the split better or guarantee that the resulting model will perform well. Complete reproducibility can also depend on the software versions, data, hardware, and other sources of randomness.

! Important

A seed is an **experimental control**, not a performance setting. It makes a particular sequence of random choices repeatable. It should not be selected because it produces the most favourable test result.

The Need for Data Splitting

Performance on the training data tells us how well a model fits examples it has already seen. It generally provides an optimistic estimate of performance on new examples, so we use a separate test set to estimate generalization. The test set should not influence training or the choices made while constructing the model. We will examine more complete evaluation procedures later in the course.

Reproducible Partitions

When examples can reasonably be treated as independent observations from the same distribution, a random partition reduces the effect of their original ordering and helps produce comparable subsets. It does not guarantee that each subset will be representative, particularly when the dataset is small. Time-series, grouped, or otherwise dependent data require different splitting strategies.

First, we create a toy dataset with 10 examples. We use colour names to make it easy to track where each example ends up. `train_test_split` can partition these strings directly, although most estimators require features to be represented numerically before training.

```
X = [['red'], ['blue'], ['green'], ['yellow'], ['purple'],
      ['orange'], ['pink'], ['brown'], ['black'], ['white']]

# y contains binary targets (0 or 1)

y = [0, 1, 0, 1, 0, 1, 0, 1, 0, 1]

print("Original examples (feature, target):")
print([(item[0], target) for item, target in zip(X, y)])
```

Original examples (feature, target):

```
[('red', 0), ('blue', 1), ('green', 0), ('yellow', 1), ('purple', 0), ('orange', 1), ('pink', 0), ('brown', 1), ('black', 0), ('white', 1)]
```

Next, we use scikit-learn's `train_test_split` function to partition `X` and `y` together. Corresponding features and targets therefore remain paired. We produce five splits by providing five seeds to `random_state`. The argument `stratify=y` asks the function to preserve the class proportions as closely as the subset sizes permit.

```
from sklearn.model_selection import train_test_split

seeds = [1, 42, 100, 2024, 9999]

for seed in seeds:

    # random_state controls the shuffling applied to the data before the split

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.4, random_state=seed, stratify=y
    )

    # Keep each colour paired with its target for easier inspection

    train_examples = [(item[0], target) for item, target in zip(X_train, y_train)]
    test_examples = [(item[0], target) for item, target in zip(X_test, y_test)]

    print(f"Seed (random_state) = {seed:<4}")
    print(f"  Training set: {train_examples}")
    print(f"  Test set:      {test_examples}\n")
```

Seed (random_state) = 1

```
Training set: [('black', 0), ('blue', 1), ('orange', 1), ('green', 0), ('white', 1), ('purple', 1)]
Test set:      [('yellow', 1), ('brown', 1), ('red', 0), ('pink', 0)]
```

Seed (random_state) = 42

```
Training set: [('brown', 1), ('black', 0), ('green', 0), ('yellow', 1), ('orange', 1), ('purple', 1)]
Test set:      [('red', 0), ('pink', 0), ('blue', 1), ('white', 1)]
```

Seed (random_state) = 100

```
Training set: [('brown', 1), ('blue', 1), ('green', 0), ('purple', 0), ('yellow', 1), ('pink', 1)]
Test set:      [('orange', 1), ('red', 0), ('black', 0), ('white', 1)]
```

Seed (random_state) = 2024

```
Training set: [('yellow', 1), ('white', 1), ('black', 0), ('green', 0), ('orange', 1), ('purple', 1)]
Test set:      [('blue', 1), ('brown', 1), ('red', 0), ('purple', 0)]
```

```
Seed (random_state) = 9999
Training set: [('red', 0), ('brown', 1), ('black', 0), ('blue', 1), ('purple', 0), ('orange', 1)]
Test set:      [('yellow', 1), ('white', 1), ('green', 0), ('pink', 0)]
```

In these examples, changing the seed changes which examples belong to each subset. Passing the same integer to `random_state` reproduces the same partition, provided the data and software environment remain unchanged.

Conclusion: Reproducibility vs. Variability

It is important to distinguish between reproducing one experiment and measuring sensitivity to random choices.

Setting a seed such as `random_state=42` lets peers reproduce a particular partition. However, one partition may be unusually favourable or unfavourable, so a reproducible result is not necessarily a reliable summary of expected performance.

Later in the course, we will use repeated evaluation procedures to measure this variability. Such experiments should use a predetermined procedure and report both typical performance and its spread, rather than trying many seeds and retaining the best result. The [lecture](#) illustrates the underlying phenomenon: different partitions can produce different decision trees and different measured performance.

Documentation

- [Python random module](#)
- [scikit-learn train_test_split](#)
- [Controlling randomness in scikit-learn](#)

References

- Bethard, S. (2022). “We need to talk about random seeds”. arXiv preprint [arXiv:2210.13393](#).
- Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). “Deep reinforcement learning that matters”. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1).
- Bouthillier, X., Delaunay, P., Bronzi, M., et al. (2021). “Accounting for variance in machine learning benchmarks”. *Proceedings of Machine Learning and Systems*, 3, 747-769.